Technical Report

Gradient Estimation for Mixture Distributions

Abstract: Finite mixture distributions provide flexible posterior approximations whose optimization requires gradients through both component parameters and mixture weights. We review the main approaches to this problem and describe four unbiased estimators: score-function, stratified, implicit reparameterization, and post-stratified. A running example connects their sampling procedures and gradients, and a comparison explains their variance and computational tradeoffs, including the Sticking the Landing modification for variational inference. These estimators will form ReMix, a PyTorch library that separates mixture distributions, objectives, and gradient estimation so that methods can be exchanged within the same optimization loop.

1 Introduction

Variational inference approximates a posterior by optimizing a tractable density. Finite mixtures make the approximation more expressive by representing several modes with separate components. Variational boosting adds these components incrementally [1], while automatic differentiation variational inference with mixtures jointly optimizes their parameters and weights [2].

Sampling a mixture involves drawing a categorical index and then a continuous value from the chosen component. Differentiating the component sample captures how that value changes with the component parameters. Learning mixture weights also requires accounting for changes in the probability of choosing each component. Score-function estimators use derivatives of the log density [3], while pathwise estimators differentiate the sampled value, as in variational autoencoders [4, 5]. Each approach has its own variance and computational requirements [6].

Several methods adapt these ideas to mixtures. Stratification samples every component and combines the resulting gradients using the mixture weights [2, 7]. Implicit reparameterization obtains sample derivatives from distribution functions [8, 9]. Transport-based methods describe how probability mass moves as parameters change [10, 11]. This perspective also leads to a post-stratified estimator, which averages component derivatives at a single mixture sample. Burroni and Sheldon [12] bring these approaches into a common formulation and analyze their computational and variance properties.

Our goal is to collect these four algorithms in ReMix, a convenient PyTorch library for optimizing mixture distributions and comparing gradient estimators. Its shared interface separates the mixture, the objective, and the estimator, allowing the same model and training loop to be reused across methods. A proof-of-concept implementation already provides this structure and a score-function baseline. The next steps are to add the remaining estimators for mixtures of diagonal Gaussian components and the Sticking the Landing (STL) modification for evidence lower bound gradients [7]. This report develops the estimators through a common example and explains the tradeoffs that the library will make accessible through this interface.

2 Related Work

Methods for discrete random variables provide further ways to handle a mixture's categorical selector. Concrete and Gumbel–Softmax use a smooth, temperature-controlled approximation and differentiate

the resulting relaxed objective [13, 14]. REBAR uses a continuous relaxation as a control variate with a correction that preserves unbiasedness for the discrete objective [15]. RELAX learns a differentiable control variate jointly with the model [16]. ARSM constructs coupled categorical samples through augmentation and category swaps, sharing randomness to reduce gradient variance [17]. These approaches offer alternatives for optimizing mixture selection probabilities.

Other work extends pathwise differentiation to broader continuous families. Generalized reparameterization combines a sample derivative with a score correction for parameter-dependent transformed noise [18]. Naesseth et al. [19] derive gradients through acceptance–rejection samplers, enabling applications to distributions such as Gamma and Dirichlet. These methods provide useful directions for extending the component families supported by ReMix. For the initial implementation, an existing PyTorch package for implicit mixture gradients provides a software reference [20]; the discussion of mixture sampling derivatives in PyTorch issue #39178 gives additional implementation context [21].

3 Problem Setup

Let $\theta \in \mathbb{R}^m$ contain the parameters of a mixture density on $\mathbb{R}^d$. Our objective and mixture are

$$\mathcal{J}(\theta) = \mathbb{E}_{q_\theta}[f_\theta(Z)], \qquad q_\theta(z) = \sum_{k=1}^{K} \pi_k(\theta) q_{k,\theta}(z), \qquad \pi_k > 0, \quad \sum_k \pi_k = 1. \tag{1}$$

We assume differentiable densities that are positive on the interior of a common, parameter-independent support, with integrable derivatives allowing differentiation under the integral. Pathwise identities additionally require vanishing boundary flux; variance comparisons require finite second moments.

The responsibility $r_k(z) = \pi_k q_{k,\theta}(z)/q_\theta(z)$ is the conditional probability that component k generated z. We write $a_k = \nabla_\theta \log \pi_k$ for the weight score and $s_\theta(z) = \partial_\theta \log q_\theta(z)$ for the mixture score. For the objective, $h_\theta(z) = \nabla_z f_\theta(z)$ describes its sensitivity to the sample, while $d_\theta(z) = \partial_\theta f_\theta(z)$ captures direct parameter dependence at fixed z.

Component samples take the form $Z_k = T_{k,\theta}(\varepsilon_k)$, where $\varepsilon_k \sim \rho$ has a parameter-independent distribution. Stratification uses derivatives of these maps. Post-stratification also uses their inverses; for this method, we assume differentiable component bijections onto the common support and write $V_k(z)$ for $D_\theta T_{k,\theta}$ evaluated at $\varepsilon_k = T_{k,\theta}^{-1}(z)$.

For a fixed model $p(x, z)$, variational inference maximizes the evidence lower bound (ELBO), with integrand

$$f_\theta(z) = \log p(x, z) - \log q_\theta(z), \qquad d_\theta(z) = -s_\theta(z). \tag{2}$$

The score has zero expectation, so omitting d_θ preserves the expected ELBO gradient. This is the STL modification [7]. In automatic differentiation, it freezes the parameters of q inside the log-density term while preserving its derivative through z. We denote an operation that preserves a value and stops its derivative by sg. For general objectives, the estimators retain d_θ.

As a running example, consider a mixture of two univariate Gaussians, $q_\theta(z) = \pi\mathcal{N}(z; \mu_1, \sigma_1^2) + (1 - \pi)\mathcal{N}(z; \mu_2, \sigma_2^2)$, where $\pi = \text{sigmoid}(\alpha)$ and $\sigma_k > 0$. We use the objective $f(z) = -z^2/2$, which favors samples close to zero. Writing $m_k = -(\mu_k^2 + \sigma_k^2)/2$ for its expectation in component k, we obtain

$$\mathcal{J} = \pi m_1 + (1 - \pi) m_2, \qquad \partial_\alpha \mathcal{J} = \pi(1 - \pi)(m_1 - m_2), \qquad \partial_{\mu_k} \mathcal{J} = -\pi_k \mu_k, \tag{3}$$

where $\pi_1 = \pi$ and $\pi_2 = 1 - \pi$. Thus, ascent moves each mean toward zero and shifts weight toward the component with the larger expected objective. The known gradient lets us follow how each estimator recovers the same quantity from different random samples. Here $h(z) = -z$ and $d(z) = 0$; the ELBO additionally has the direct density derivative described above.

4 Gradient Estimators

The following estimators target $\nabla_\theta \mathcal{J}$ under the assumptions above, with optional STL for the ELBO.

Algorithms 1–4 return an objective estimate and a scalar surrogate $\mathcal{S}$; differentiating $\mathcal{S}$ gives the estimated ascent gradient. The first output is detached for logging. For a loss-minimizing optimizer, the training loss is $-\mathcal{S}$, corresponding to `surrogate_loss` in the library interface. Write $\varphi_\theta(z) = f_\theta(z)$ for ordinary differentiation. With STL, use $\varphi_\theta(z) = \log p(x, z) - \log q_{\mathrm{sg}(\theta)}(z)$, retaining the derivative through z. For diagonal Gaussian components, $\rho = \mathcal{N}(0, I)$ and $T_{k,\theta}(\varepsilon) = \mu_k + \sigma_k \odot \varepsilon$. The single-sample algorithms first select a component with probabilities π, then draw its continuous value. Independent repetitions can be averaged to increase the sample budget.

4.1 Score-function estimator

The score-function method differentiates the mixture density at a fixed sample. For $Z \sim q_\theta$ and a baseline b independent of Z, it gives [3, 6]

$$\widehat{g}_{\mathrm{SF}}(Z) = (f_\theta(Z) - b)s_\theta(Z) + d_\theta(Z). \tag{4}$$

The identity follows by differentiating under the integral. The baseline can be chosen to reduce variance because its contribution has zero expectation. The method requires a differentiable mixture log density.

Algorithm 1 draws one component and its base noise, then detaches the resulting value. Differentiating the log density supplies the score term, while differentiating $\varphi_\theta(z_0)$ supplies the direct objective term. The baseline is detached in the score coefficient. With softmax logits α and component parameters independent of α, the weight gradient is $(f_\theta(Z) - b)(r_j(Z) - \pi_j)$ for logit α_j with STL.

In the running example, draw $c \sim \mathrm{Cat}(\pi, 1 - \pi)$ and $\varepsilon \sim \mathcal{N}(0, 1)$, giving $z = \mu_c + \sigma_c \varepsilon$. The logit estimate is $(f(z) - b)(r_1(z) - \pi)$, while the estimate for mean μ_k is $(f(z) - b)r_k(z)(z - \mu_k)/\sigma_k^2$. Both use the objective value to scale a density derivative. Their expectations give the derivatives in Eq. 3, with the baseline controlling how objective values are centered.

Algorithm 1: Score-function sampling and surrogate

Input: Mixture q_θ, component maps $T_{k,\theta}$, objective φ_θ, baseline b
Output: $(\widehat{\mathcal{J}}, \mathcal{S})$: detached objective estimate for logging and ascent surrogate, $\nabla_\theta \mathcal{S} = \widehat{g}_{\mathrm{SF}}$

Draw $c \sim \mathrm{Cat}(\pi)$ and $\varepsilon \sim \rho$;
$z_0 \leftarrow \mathrm{sg}(T_{c,\theta}(\varepsilon))$;
$v \leftarrow \varphi_\theta(z_0)$; $\ell \leftarrow \log q_\theta(z_0)$;
$\mathcal{S} \leftarrow \mathrm{sg}(v - b)\,\ell + v$;
return $(\mathrm{sg}(v), \mathcal{S})$;

4.2 Stratified estimator

Stratification evaluates the objective within every component and combines the results using the mixture weights. Draw $N_k \geq 1$ independent base samples for component k, with each N_k fixed, and set $Z_{k,n} = T_{k,\theta}(\varepsilon_{k,n})$. Differentiating the weighted component expectations gives [2, 7]

$$\widehat{g}_{\mathrm{ST}} = \sum_k \frac{\pi_k}{N_k} \sum_{n=1}^{N_k} \left[f_\theta(Z_{k,n}) a_k + D_\theta T_{k,\theta}(\varepsilon_{k,n})^\top h_\theta(Z_{k,n}) + d_\theta(Z_{k,n}) \right]. \tag{5}$$

In practice, automatic differentiation of the weighted sample average constructs all three terms. Algorithm 2 draws independent noise within every component and retains the derivative through each transformation and the outer mixture weights. Every sampled point is evaluated using the full mixture density in the ELBO.

Algorithm 2: Stratified sampling and surrogate

Input: Weights π, component maps $T_{k,\theta}$, objective φ_θ, fixed counts $N_k \geq 1$
Output: $(\widehat{\mathcal{J}}, \mathcal{S})$: detached objective estimate for logging and ascent surrogate, $\nabla_\theta \mathcal{S} = \widehat{g}_{\mathrm{ST}}$

$\mathcal{S} \leftarrow 0$;
for $k = 1, \ldots, K$ **do**
  for $n = 1, \ldots, N_k$ **do**
    Draw independent $\varepsilon_{k,n} \sim \rho$;
    $z_{k,n} \leftarrow T_{k,\theta}(\varepsilon_{k,n})$;
    $\mathcal{S} \leftarrow \mathcal{S} + (\pi_k / N_k)\varphi_\theta(z_{k,n})$;
  end
end
return $(\mathrm{sg}(\mathcal{S}), \mathcal{S})$;

Sampling every component removes component-selection randomness and allows the sample budget to be allocated across components. One sample per component requires K objective evaluations. For a chain of T conditional mixtures, full enumeration requires $\prod_{t=1}^{T} K_t$ evaluations, where K_t is the component count at layer t. This growth motivates single-sample methods.

For the two-Gaussian example, stratification draws independent $z_1 = \mu_1 + \sigma_1\varepsilon_1$ and $z_2 = \mu_2 + \sigma_2\varepsilon_2$. With one sample per component, the logit estimate becomes $\pi(1-\pi)(f(z_1) - f(z_2))$, and the mean estimates are $-\pi z_1$ and $-(1-\pi)z_2$. Each update directly compares both components, using two objective evaluations. Averaging these expressions recovers the component expectations and mean gradients in Eq. 3.

4.3 Implicit reparameterization

Implicit reparameterization differentiates an exact mixture sample through a standardization S_θ, a differentiable bijection mapping $Z \sim q_\theta$ to a fixed reference distribution. Holding the standardized value constant gives [8, 9]

$$A_\theta(z)V_{\mathrm{IR}}(z) = -D_\theta S_\theta(z), \qquad A_\theta(z) = D_z S_\theta(z), \qquad \widehat{g}_{\mathrm{IR}}(Z) = V_{\mathrm{IR}}(Z)^\top h_\theta(Z) + d_\theta(Z). \tag{6}$$

The matrix A_θ must be nonsingular, and the inverse transformation must satisfy the differentiation assumptions. In one dimension, S_θ is the mixture cumulative distribution function (CDF), so the sample derivative is $-\partial_\theta F_\theta(z)/q_\theta(z)$.

For a multivariate mixture, a suitable standardization is the Rosenblatt transform: coordinate i is the conditional CDF $S_{\theta,i}(z) = F_\theta(z_i \mid z_{<i})$. Its component weights are proportional to $\pi_k q_{k,\theta}(z_{<i})$, accounting for the information in preceding coordinates. For factorized components, the prefix density is a product of univariate densities, and the component conditional CDF is its univariate CDF. The resulting Jacobian is lower triangular with positive conditional densities on the diagonal, enabling a triangular solve.

Algorithm 3 first draws an exact sample through the categorical index and component map. It then evaluates the standardization at that detached point and attaches its implicit derivative through a linear solve. The corrected value $\widetilde{z}$ equals z_0 and carries the first-order implicit gradient. The method requires one objective evaluation, CDF derivatives, and a Jacobian solve. A generic reverse-mode construction costs $O(dC_S + d^2)$, where C_S is the cost of evaluating S_θ.

In the running example, let F_k denote the CDF of component k. The standardization is $F_\theta(z) = \pi F_1(z) + (1-\pi)F_2(z)$, giving $\partial_\alpha z = -\pi(1-\pi)(F_1(z) - F_2(z))/q_\theta(z)$ and $\partial_{\mu_k} z = r_k(z)$. Multiplying by $h(z) = -z$ yields the objective gradients. The weight derivative describes how the sampled location moves when the mixture weight changes while its cumulative probability stays fixed. This uses one objective evaluation and the CDFs of both components.

Algorithm 3: Implicit sampling and surrogate

Input: Weights π, component maps $T_{k,\theta}$, standardization S_θ, objective φ_θ
Output: $(\widehat{\mathcal{J}}, \mathcal{S})$: detached objective estimate for logging and ascent surrogate, $\nabla_\theta \mathcal{S} = \widehat{g}_{\mathrm{IR}}$

Draw $c \sim \mathrm{Cat}(\pi)$ and $\varepsilon \sim \rho$;
$z_0 \leftarrow \mathrm{sg}(T_{c,\theta}(\varepsilon))$;
$s \leftarrow S_\theta(z_0)$; $\quad A \leftarrow \mathrm{sg}(D_z S_\theta(z)|_{z=z_0})$;
Solve the triangular system $Au = -s$, retaining the derivative through s;
$\widetilde{z} \leftarrow z_0 + u - \mathrm{sg}(u)$;
$v \leftarrow \varphi_\theta(\widetilde{z})$; $\quad \mathcal{S} \leftarrow v$;
return $(\mathrm{sg}(v), \mathcal{S})$;

4.4 Post-stratified estimator

Post-stratification draws one exact mixture sample and evaluates how every component could have generated that value. It recovers the base noise through each inverse map and averages the component derivatives using responsibilities:

$$\overline{V}(z) = \sum_k r_k(z) V_k(z), \qquad \widehat{g}_{\mathrm{PS}}(Z) = \overline{V}(Z)^\top h_\theta(Z) + f_\theta(Z) \sum_k r_k(Z) a_k + d_\theta(Z). \tag{7}$$

The identity $\mathbb{E}_{q_\theta}[r_k(Z)H(Z)] = \pi_k \mathbb{E}_{q_{k,\theta}}[H(Z)]$ connects this expression to the stratified gradient and establishes unbiasedness [11, 12]. Component parameters receive pathwise derivatives, and trainable weights receive the score contribution. With independent softmax logits, the latter is $f_\theta(Z)(r_j(Z) - \pi_j)$.

Returning to the example, post-stratification draws one value z and recovers $e_k = (z - \mu_k)/\sigma_k$ for both components. Its mean estimate is $-r_k(z)z$, which coincides with the implicit estimate in this univariate Gaussian case. Its logit estimate is $f(z)(r_1(z) - \pi)$, the same as the score-function estimate with zero baseline. The example separates the two roles of the method: responsibility-weighted sample derivatives update the component parameters, while a score term updates the mixture weights.

Algorithm 4 draws one component to obtain z_0, then recovers a base-noise value for every component at that same point. For diagonal Gaussians, the inverse map is $T_{k,\theta}^{-1}(z) = (z - \mu_k) \oslash \sigma_k$. The recovered noises e_k and responsibilities $\bar{r}_k$ are detached before reconstructing the sample. The resulting surrogate is

$$\widetilde{z} = \sum_k \bar{r}_k T_{k,\theta}(e_k), \qquad \mathcal{S}_{\mathrm{PS}} = f_\theta(\widetilde{z}) + \mathrm{sg}(f_\theta(z_0)) \sum_k \bar{r}_k \log \pi_k. \tag{8}$$

Each reconstructed component sample equals z_0, so $\widetilde{z}$ preserves the original value while carrying the averaged derivative. The objective value is reused in the detached weight correction. The choice of φ_θ applies ordinary differentiation or STL to the first term.

Algorithm 4: Post-stratified sampling and surrogate

Input: Mixture q_θ, component maps $T_{k,\theta}$ and their inverses, objective φ_θ
Output: $(\widehat{\mathcal{J}}, \mathcal{S})$: detached objective estimate for logging and ascent surrogate, $\nabla_\theta \mathcal{S} = \widehat{g}_{\mathrm{PS}}$

Draw $c \sim \mathrm{Cat}(\pi)$ and $\varepsilon \sim \rho$;
$z_0 \leftarrow \mathrm{sg}(T_{c,\theta}(\varepsilon))$;
for $k = 1, \ldots, K$ **do**
  $e_k \leftarrow \mathrm{sg}(T_{k,\theta}^{-1}(z_0))$; $\quad \bar{r}_k \leftarrow \mathrm{sg}(\pi_k q_{k,\theta}(z_0)/q_\theta(z_0))$;
end
$\widetilde{z} \leftarrow \sum_k \bar{r}_k T_{k,\theta}(e_k)$;
$v \leftarrow \varphi_\theta(\widetilde{z})$;
$\mathcal{S} \leftarrow v + \mathrm{sg}(v) \sum_k \bar{r}_k \log \pi_k$;
return $(\mathrm{sg}(v), \mathcal{S})$;

This conditional averaging is a Rao–Blackwellization: it has a smaller or equal covariance than using only the sampled component's pathwise and weight-score terms, including with STL [12]. It requires one objective evaluation, with all component densities and inverse maps evaluated at the same sample.

5 Comparing the Estimators

The running example shows how all four methods recover the same expected gradient while using different information from the mixture. Score-function estimation weights density derivatives by the observed objective. Stratification samples every component and differentiates their weighted objectives. Implicit reparameterization moves one sampled point while keeping its standardized value fixed. Post-stratification uses one point to average the sample derivatives of all components, retaining a score term for the weights. These differences determine both the randomness in an update and the work needed to compute it.

For score-function estimation, the objective value scales the entire density score, making the baseline an important part of the comparison. Stratification removes the categorical draw by assigning samples to every component; its remaining randomness comes from sampling within those components. This also ensures that a component with a small mixture weight receives samples in each update. Post-stratification instead averages the component derivatives conditional on one sampled value. Its Rao–Blackwell guarantee compares it with the corresponding estimator that uses only the selected component's pathwise and weight-score terms [12]. Comparing it with full stratification or implicit reparameterization requires examining the objective, the mixture, and the sample budget. In the running example, implicit and post-stratified estimates agree for the means but differ for the mixture weight, so even within one problem the comparison depends on which parameters are being optimized.

The available operations also affect the choice. Table 1 summarizes one update of each method. Score-function estimation needs sampling and mixture log-density derivatives, making it a useful baseline for a broad range of component families. Stratification adds component sampling derivatives and permits separate sample counts for each component. Post-stratification uses inverse component maps to obtain averaged sample derivatives with one objective evaluation. Implicit reparameterization also uses one objective evaluation and carries weight derivatives through the sample; its additional requirements are conditional CDFs, their derivatives, and a Jacobian solve.

Tab. 1: Requirements per update. Stratification uses one sample per component; the other methods use one mixture sample.

Estimator	Objective evaluations	Required operations
Score-function	1	Mixture log-density derivatives
Stratified	K	Component sampling derivatives
Implicit	1	Conditional CDFs and a Jacobian solve
Post-stratified	1	Inverse component maps and responsibilities

Objective counts are particularly informative when evaluating the model is expensive. In that setting, the single-sample methods can spend the saved model evaluations on additional updates or independent samples. Density computations also matter: evaluating the ELBO at one point requires the full mixture density. For diagonal Gaussians, this entails $O(Kd)$ component-density work, so evaluating the ELBO once in every component entails $O(K^2 d)$ such work. Post-stratification evaluates all component densities and inverse maps at a single point, while implicit reparameterization adds the standardization derivatives and solve described above. Vectorization and the relative cost of the model determine how these operation counts translate into runtime.

Consequently, an empirical comparison should report both gradient variance at a fixed mixture and optimization progress under matched computational budgets. Equal objective-evaluation budgets and equal wall-clock budgets answer different practical questions, especially as the number of components and the event dimension grow. STL preserves the expected ELBO gradient and changes the sample-level estimator, so its effect should be evaluated separately for each method. ReMix will expose the sample budget and STL option through a shared interface, making these comparisons possible with the same model and optimizer.

6 The ReMix Library

ReMix organizes mixture optimization around three objects: a distribution, an objective, and a gradient estimator. The proof of concept in the repository's `PoC` branch already implements this separation. Its `Mixture` object accepts mixture logits and compatible PyTorch distribution objects and provides sampling, log densities, and responsibilities. Component families can be combined when they have the same event shape and base measure. The included examples use Gaussian, Laplace, and mixed Gaussian–Laplace components, illustrating how the same interface can accommodate different choices of distribution.

The user supplies the model through a callable that evaluates $\log p(x, z)$. An `ELBOObjective` combines this value with the mixture log density, while a `GradientEstimator` constructs the update. Calling `estimate(q, objective)` returns an `EstimateResult` containing a detached `value` for logging and a scalar `surrogate_loss` for training. The latter is the negative ascent surrogate from the algorithms above, so the usual PyTorch sequence of clearing gradients, calling `backward()`, and stepping the optimizer applies directly. Keeping the logged value separate from the training surrogate makes it possible to track the ELBO consistently across estimators whose surrogate values have different forms.

To illustrate the current interface, consider fitting the weight of a two-Gaussian mixture while keeping its component means and scales fixed. Listing 1 creates the components and a target mixture with weights 0.2 and 0.8. Passing the target's log density to `ELBOObjective` gives the integrand $\log p(z) - \log q_\theta(z)$, whose expectation is $-\mathrm{KL}(q_\theta \| p)$. Maximizing it therefore fits the variational mixture to the target. In a posterior-inference application, the same callable would evaluate the model's joint log density for fixed observations.

```
import torch
from torch.distributions import Normal

from remix import ELBOObjective, Mixture, ScoreFunction

components = [Normal(-2.0, 1.0), Normal(2.0, 1.0)]
target = Mixture(torch.tensor([0.2, 0.8]).log(), components)
logits = torch.nn.Parameter(torch.zeros(2))

objective = ELBOObjective(target.log_prob)
estimator = ScoreFunction(n_samples=512, baseline=0.0)
optimizer = torch.optim.Adam([logits], lr=0.05)
```

Listing 1: Setting up a mixture-fitting objective with the current PoC API.

The optimization loop in Listing 2 rebuilds the mixture from its current logits at each step, obtains the surrogate loss, and uses an ordinary PyTorch optimizer update. The detached value reports the estimated ELBO before that update. The final softmax converts the learned logits into mixture probabilities. This example uses the implemented score-function estimator; its sample count and baseline are configured independently of the model and optimization loop.

```
for step in range(300):
    optimizer.zero_grad()
    q = Mixture(logits, components)
    result = estimator.estimate(q, objective)
    result.surrogate_loss.backward()
    optimizer.step()

    if step % 50 == 0:
        print(step, result.value.item())

print(logits.detach().softmax(dim=-1))
```

Listing 2: Optimizing the mixture weights and logging the objective.

This structure makes replacing an estimator a local choice in a training script: the mixture parameters, model callable, optimizer, and logging code can be retained. The proof of concept currently provides score-function estimation with a fixed baseline and a configurable sample count. The planned extensions will add stratified, implicit, and post-stratified estimators, initially for diagonal Gaussian components, together with optional STL for the ELBO. Each estimator will use the distribution operations it needs, such as component sampling maps, inverse maps, or conditional CDFs. Shared density and responsibility calculations will use log-sum-exp and normalized component log densities, while implicit derivatives will use triangular solves.

The resulting library will support comparisons of gradient variance, ELBO progress, runtime, and memory use across component counts, overlap levels, and event dimensions. Its practical purpose is to let users carry the mathematical choices described in this report into an ordinary optimization loop and compare them under the computational budgets discussed in Section 5.

References

[1] Andrew C. Miller, Nicholas J. Foti, and Ryan P. Adams. Variational boosting: Iteratively refining posterior approximations. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 2420–2429, 2017. URL https://proceedings.mlr.press/v70/miller17a.html.

[2] Warren Morningstar, Sharad Vikram, Cusuh Ham, Andrew Gallagher, and Joshua Dillon. Automatic differentiation variational inference with mixtures. In *Proceedings of the 24th International Conference on Artificial Intelligence and Statistics*, volume 130 of *Proceedings of Machine Learning Research*, pages 3250–3258, 2021. URL https://proceedings.mlr.press/v130/morningstar21b.html.

[3] Rajesh Ranganath, Sean Gerrish, and David M. Blei. Black box variational inference. In *Proceedings of the Seventeenth International Conference on Artificial Intelligence and Statistics*, volume 33 of *Proceedings of Machine Learning Research*, pages 814–822, 2014. URL https://proceedings.mlr.press/v33/ranganath14.html.

[4] Diederik P. Kingma and Max Welling. Auto-encoding variational Bayes. In *International Conference on Learning Representations*, 2014. URL https://arxiv.org/abs/1312.6114.

[5] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. Stochastic backpropagation and approximate inference in deep generative models. In *Proceedings of the 31st International Conference on Machine Learning*, volume 32(2) of *Proceedings of Machine Learning Research*, pages 1278–1286, 2014. URL https://proceedings.mlr.press/v32/rezende14.html.

[6] Shakir Mohamed, Mihaela Rosca, Michael Figurnov, and Andriy Mnih. Monte Carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132):1–62, 2020. URL https://www.jmlr.org/papers/v21/19-346.html.

[7] Geoffrey Roeder, Yuhuai Wu, and David K. Duvenaud. Sticking the landing: Simple, lower-variance gradient estimators for variational inference. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1703.09194.

[8] Alex Graves. Stochastic backpropagation through mixture density distributions, 2016. URL https://arxiv.org/abs/1607.05690.

[9] Michael Figurnov, Shakir Mohamed, and Andriy Mnih. Implicit reparameterization gradients. In *Advances in Neural Information Processing Systems*, volume 31, 2018. URL https://arxiv.org/abs/1805.08498.

[10] Martin Jankowiak and Fritz Obermeyer. Pathwise derivatives beyond the reparameterization trick. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pages 2235–2244, 2018. URL https://proceedings.mlr.press/v80/jankowiak18a.html.

[11] Martin Jankowiak and Theofanis Karaletsos. Pathwise derivatives for multivariate distributions. In *Proceedings of the Twenty-Second International Conference on Artificial Intelligence and Statistics*, volume 89 of *Proceedings of Machine Learning Research*, pages 333–342, 2019. URL https://proceedings.mlr.press/v89/jankowiak19a.html.

[12] Javier Burroni and Daniel Sheldon. Gradient estimation for mixture variational inference. *Journal of Machine Learning Research*, 27(191):1–34, 2026. URL https://www.jmlr.org/papers/v27/25-2560.html.

[13] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. The concrete distribution: A continuous relaxation of discrete random variables. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1611.00712.

[14] Eric Jang, Shixiang Gu, and Ben Poole. Categorical reparameterization with Gumbel–Softmax. In *International Conference on Learning Representations*, 2017. URL https://arxiv.org/abs/1611.01144.

[15] George Tucker, Andriy Mnih, Chris J. Maddison, Dieterich Lawson, and Jascha Sohl-Dickstein. REBAR: Low-variance, unbiased gradient estimates for discrete latent variable models. In *Advances in Neural Information Processing Systems*, volume 30, 2017. URL https://arxiv.org/abs/1703.07370.

[16] Will Grathwohl, Dami Choi, Yuhuai Wu, Geoffrey Roeder, and David Duvenaud. Backpropagation through the void: Optimizing control variates for black-box gradient estimation. In *International Conference on Learning Representations*, 2018. URL https://arxiv.org/abs/1711.00123.

[17] Mingzhang Yin, Yuguang Yue, and Mingyuan Zhou. ARSM: Augment-REINFORCE-swap-merge estimator for gradient backpropagation through categorical variables. In *Proceedings of the 36th International Conference on Machine Learning*, volume 97 of *Proceedings of Machine Learning Research*, pages 7095–7104, 2019. URL https://proceedings.mlr.press/v97/yin19c.html.

[18] Francisco J. R. Ruiz, Michalis K. Titsias, and David M. Blei. The generalized reparameterization gradient. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL https://arxiv.org/abs/1610.02287.

[19] Christian A. Naesseth, Francisco J. R. Ruiz, Scott W. Linderman, and David M. Blei. Reparameterization gradients through acceptance-rejection sampling algorithms. In *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics*, volume 54 of *Proceedings of Machine Learning Research*, pages 489–498, 2017. URL https://proceedings.mlr.press/v54/naesseth17a.html.

[20] vsimkus. torch-reparametrised-mixture-distribution. GitHub repository, n.d. URL https://github.com/vsimkus/torch-reparametrised-mixture-distribution. Accessed 21 October 2026.

[21] PyTorch contributors. Add support for `rsample` to MixtureSameFamily distribution. GitHub issue #39178, 2020. URL https://github.com/pytorch/pytorch/issues/39178. Accessed 21 October 2026.